Docker Tutorial

Anthony Baire

Université de Rennes 1 / UMR IRISA

October 22, 2026



This tutorial is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs 3.0 France License

Summary

1. Introduction
2. Managing docker containers
3. Inputs/Outputs
4. Managing docker images
5. Building docker images
6. Security considerations
7. The ecosystem & the future

Part 1.
Introduction

What is Docker (1/3)



"Docker is an open platform for developers and sysadmins to build, ship, and run distributed applications. Consisting of Docker Engine, a portable, lightweight runtime and packaging tool, and Docker Hub, a cloud service for sharing applications and automating workflows, Docker enables apps to be quickly assembled from components and eliminates the friction between development, QA, and production environments. As a result, IT can ship faster and run the same app, unchanged, on laptops, data center VMs, and any cloud."

source: `https://www.docker.com/whatisdocker/`

What is Docker (2/3)

- a container manager
  - lightweight virtualisation
    (host and guest systems share the same kernel)
  - based on linux namespaces and cgroups

- massively copy-on-write
  - immutable images
  - instant deployment
  - suitable for micro-services (one process, one container)

$\rightarrow$ immutable architecture

What is Docker (3/3)

- a build system
  - images may be built from sources
  - using a simple Domain Specific Language (DSL)[1] (`Dockerfile`)
- a set of REST APIs
  - Engine API (control the docker engine)
  - Plugin API (extend the engine $\rightarrow$ network, storage, authorisation)
  - Registry API (publish/download images)
  - Swarm API (manage a clusted of docker machines)

[1] A domain-specific language (DSL) is a computer language specialized to a specific application domain. This is in contrast to a general-purpose language (GPL), which is broadly applicable across domains. Wikipedia: Domain Specific Language. For instance, HTML is a DSL for creating web pages, while C and C++ are GPLs.

How Docker helps?

- **normalisation:** same environment (container image) for
  - development
  - jobs on the computing grid
  - continuous integration
  - peer review
  - demonstrations, tutorials
  - technology transfer

- **archival** *(ever tried to reuse old codes)*
  - source → Dockerfile = recipe to rebuild the env from scratch
  - binary → docker image = immutable snapshot of the software with its runtime environment
    → can be re-run at any time later

In practice

A docker image is an immutable snapshot of the filesystem (fs)

A docker container is

- a temporary file system
  - layered over an immutable fs (docker image)
  - fully writable (copy-on-write[2])
  - dropped at container's end of life (unless a `commit` is made)
- a network stack
  - with its own private address *(by defaut in 172.17.x.x)*
- a process group
  - one main process launched inside the container
  - all sub-processes SIGKILLed when the main process exits

[2] several possible methods: overlayfs (default), btrfs, lvm, zfs, aufs

Rootful vs. Rootless

The docker containers may be run :

- in **rootful mode (by default)**
  - the docker daemon is run by the root user
  - the user and group ids (uid/gid) inside the container are the exactly the same users and groups as on the host system
    $\rightarrow$ *the user with uid:gid 0:0 (root) inside the container is the root user of the host system*

- in **rootless mode**
  - the docker daemon is run by an ordinary user
  - the containers are configured with a **user namespace** that maps the user and group ids (uid/gid) inside the container to a range of subordinate uids/gids on the host system[3]
    $\rightarrow$ *the user with uid:gid 0:0 (root) inside the container is mapped to the first uid and gid of the range allocated to the user running the daemon*

[3] see `/etc/subuid` and `/etc/subgid`

Installation

https://docs.docker.com/engine/installation/

Official packages:

- Docker Engine (a.k.a. Docker CE) on Linux
- Docker Desktop on Linux, macOS and windows
  - GUI
  - Build, share, setup and run Docker containers
  - virtualization for Docker Engine (Windows and macOS)
  - non-free license (registration required, free of charge only for personal user or small organisations)

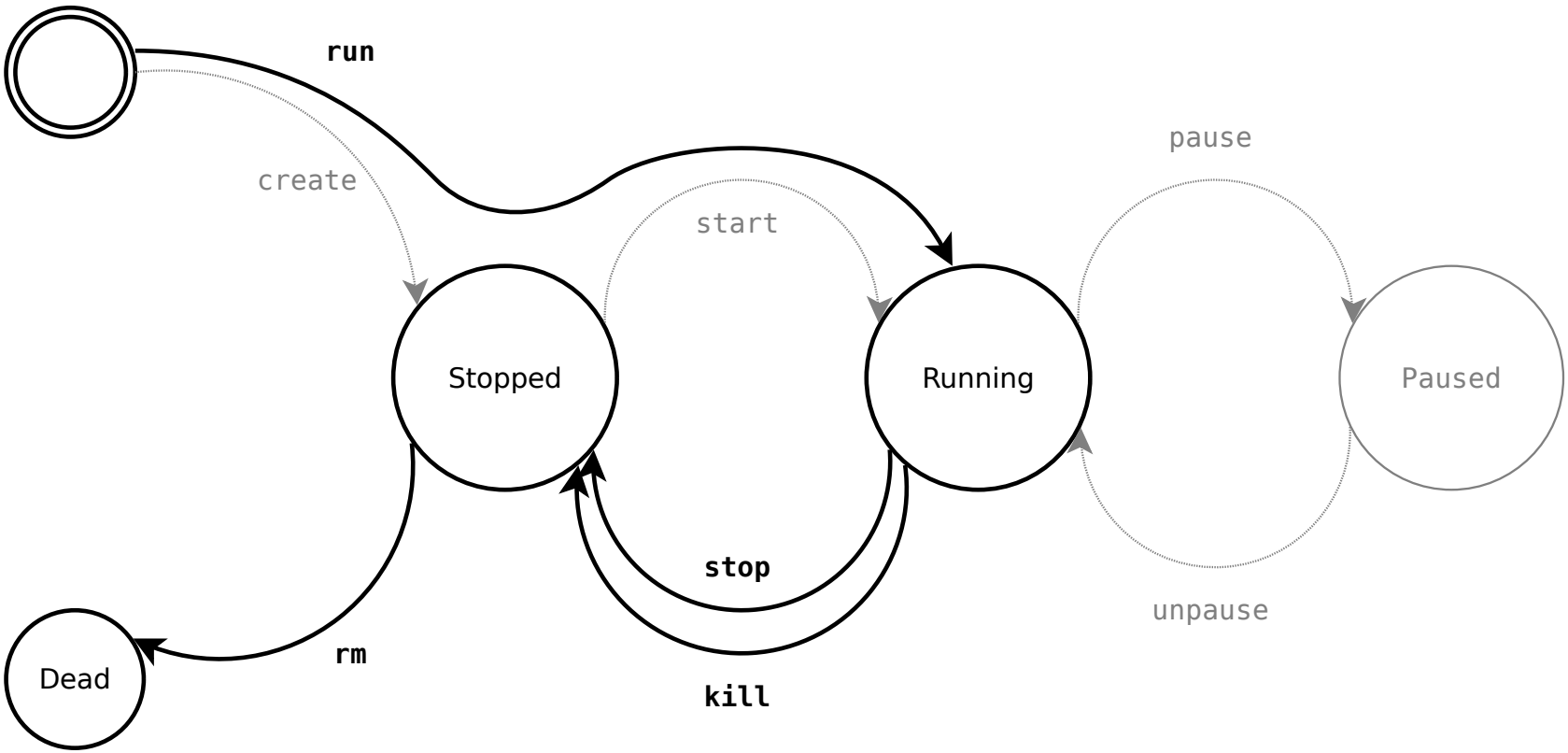
Other alternatives:

- colima (macOS, Linux)

Part 2.
Managing containers

- create/start/stop/remove containers
- inspect containers
- interact, commit new images

Lifecycle of a docker container



Container management commands

command	description
`docker create` *image* [*command*] `docker run` *image* [*command*]	create the container = `create` + `start`
`docker rename` *container new_name* `docker update` *container*	rename the container update the container config
`docker start` *container...* `docker stop` *container...* `docker kill` *container...* `docker restart` *container...*	start the container graceful[4] stop kill (SIGKILL) the container = `stop` + `start`
`docker pause` *container...* `docker unpause` *container...*	suspend the container resume the container
`docker rm` [`-f`[5]] *container...*	destroy the container

[4] send SIGTERM to the main process + SIGKILL 10 seconds later

[5] `-f` allows removing running containers (= `docker kill` + `docker rm`)

Notes about the container lifecycle

- the container filesystem is created in `docker create` and dropped in `docker rm`
  - it is persistent across `stop`/`start`
- the container configuration is mostly static
  - config is set in `create`/`run`
  - `docker update` may change only a few parameters (eg: cpu/ram/blkio allocations)
  - changing other parameters requires destroying and re-creating the container
- other commands are rather basic

```
Usage:  docker create [OPTIONS] IMAGE [COMMAND] [ARG...]

Create a new container

  -a, --attach=[]                Attach to STDIN, STDOUT or STDERR
  --add-host=[]                  Add a custom host-to-IP mapping (host:ip)
  --blkio-weight=0               Block IO (relative weight), between 10 and 1000
  --cpu-shares=0                 CPU shares (relative weight)
  --cap-add=[]                   Add Linux capabilities
  --cap-drop=[]                  Drop Linux capabilities
  --cgroup-parent=               Optional parent cgroup for the container
  --cidfile=                     Write the container ID to the file
  --cpu-period=0                 Limit CPU CFS (Completely Fair Scheduler) period
  --cpu-quota=0                  Limit CPU CFS (Completely Fair Scheduler) quota
  --cpuset-cpus=                 CPUs in which to allow execution (0-3, 0,1)
  --cpuset-mems=                 MEMs in which to allow execution (0-3, 0,1)
  --device=[]                    Add a host device to the container
  --disable-content-trust=true   Skip image verification
  --dns=[]                       Set custom DNS servers
  --dns-opt=[]                   Set DNS options
  --dns-search=[]                Set custom DNS search domains
  -e, --env=[]                   Set environment variables
  --entrypoint=                  Overwrite the default ENTRYPOINT of the image
  --env-file=[]                  Read in a file of environment variables
  --expose=[]                    Expose a port or a range of ports
  --group-add=[]                 Add additional groups to join
  -h, --hostname=                Container host name
  --help=false                   Print usage
  -i, --interactive=false        Keep STDIN open even if not attached
  --ipc=                         IPC namespace to use
  --kernel-memory=               Kernel memory limit
  -l, --label=[]                 Set meta data on a container
  --label-file=[]                Read in a line delimited file of labels
  --link=[]                      Add link to another container
  --log-driver=                  Logging driver for container
  --log-opt=[]                   Log driver options
  --lxc-conf=[]                  Add custom lxc options
  -m, --memory=                  Memory limit
  --mac-address=                 Container MAC address (e.g. 92:d0:c6:0a:29:33)
  --memory-reservation=          Memory soft limit
  --memory-swap=                 Total memory (memory + swap), '-1' to disable swap
  --memory-swappiness=-1         Tuning container memory swappiness (0 to 100)
  --name=                        Assign a name to the container
  --net=default                  Set the Network for the container
  --oom-kill-disable=false       Disable OOM Killer
  -P, --publish-all=false        Publish all exposed ports to random ports
  -p, --publish=[]               Publish a container's port(s) to the host
  --pid=                         PID namespace to use
  --privileged=false             Give extended privileges to this container
  --read-only=false              Mount the container's root filesystem as read only
  --restart=no                   Restart policy to apply when a container exits
  --security-opt=[]              Security Options
  --stop-signal=SIGTERM          Signal to stop a container, SIGTERM by default
  -t, --tty=false                Allocate a pseudo-TTY
  -u, --user=                    Username or UID (format: <name|uid>[:<group|gid>])
  --ulimit=[]                    Ulimit options
  --uts=                         UTS namespace to use
  -v, --volume=[]                Bind mount a volume
  --volume-driver=               Optional volume driver for the container
  --volumes-from=[]              Mount volumes from the specified container(s)
  -w, --workdir=                 Working directory inside the container
```

```
Usage:  docker start [OPTIONS] CONTAINER [CONTAINER...]

Start one or more stopped containers

  -a, --attach=false         Attach STDOUT/STDERR and forward signals
  --help=false               Print usage
  -i, --interactive=false    Attach container's STDIN
```

```
Usage:  docker stop [OPTIONS] CONTAINER [CONTAINER...]

Stop a running container.
Sending SIGTERM and then SIGKILL after a grace period

  --help=false       Print usage
  -t, --time=10      Seconds to wait for stop before killing it
```

```
Usage:  docker restart [OPTIONS] CONTAINER [CONTAINER...]

Restart a container

  --help=false       Print usage
  -t, --time=10      Seconds to wait for stop before killing the container
```

```
Usage:  docker kill [OPTIONS] CONTAINER [CONTAINER...]

Kill a running container

  --help=false         Print usage
  -s, --signal=KILL    Signal to send to the container
```

```
Usage:  docker rm [OPTIONS] CONTAINER [CONTAINER...]

Remove one or more containers

  -f, --force=false      Force the removal of a running container (uses SIGKILL)
  --help=false           Print usage
  -l, --link=false       Remove the specified link
  -v, --volumes=false    Remove the volumes associated with the container
```

```
Usage:  docker pause [OPTIONS] CONTAINER [CONTAINER...]

Pause all processes within a container

  --help=false    Print usage
```

`docker run` — Run a container

https://docs.docker.com/reference/run/

`docker run [ options ] image [ arg0 arg1...]`

→ create a container and start it

- the container filesystem is initialised from image *image*
- *arg0..argN* is the command run inside the container (as PID 1)

```
$ docker run debian /bin/hostname
f0d0720bd373
$ docker run debian date +%H:%M:%S
17:10:13
$ docker run debian true ; echo $?
0
$ docker run debian false ; echo $?
1
```

`docker run` — Foreground mode vs. Detached mode

- Foreground mode is the default
  - *stdout* and *stderr* are redirected to the terminal
  - `docker run` propagates the exit code of the main process
- With `-d`, the container is run in detached mode:
  - displays the ID of the container
  - returns immediately

```
$ docker run debian date
Tue Jan 20 17:32:07 UTC 2015
$ docker run -d debian date
4cbdefb3d3e1331ccf7783b32b47774fefca426e03a2005d69549f3ff06b9306
$ docker logs 4cbdef
Tue Jan 20 17:32:16 UTC 2015
```

`docker run` — TTY allocation

By default the standard streams (*stdin*, *stdout*, *stderr*) are connected through unix pipes

```
$ docker run debian ls
bin
boot
dev
...
$ docker run debian bash
$
```

With `-t` they are connected through a pseudo-terminal

```
$ docker run -t debian ls
bin   dev  home  lib64   mnt  proc  run   selinux  sys  usr
boot  etc  lib   media   opt  root  sbin  srv      tmp  var
$ docker run -t debian bash
root@10d90c09d9ac:/#
```

`docker run` — interactive mode

- By default containers are non-interactive
  - *stdin* is closed immediately
  - terminal signals are not forwarded[6]

```
$ docker run -t debian bash
root@6fecc2e8ab22:/# date
^C
$
```

- With `-i` the container runs interactively
  - *stdin* is usable
  - terminal signals are forwarded to the container

```
$ docker run -t -i debian bash
root@78ff08f46cdb:/# date
Tue Jan 20 17:52:01 UTC 2015
root@78ff08f46cdb:/# ^C
root@78ff08f46cdb:/#
```

[6] `^C` only detaches the terminal, the container keeps running in background

`docker run` — override defaults (1/2)

user (`-u`)

```
$ docker run debian whoami
root
$ docker run -u nobody debian whoami
nobody
```

working directory (`-w`)

```
$ docker run debian pwd
/
$ docker run -w /opt debian pwd
/opt
```

`docker run` — override defaults (2/2)

environment variables (`-e`)

```
$ docker run debian sh -c 'echo $FOO $BAR'

$ docker run -e FOO=foo -e BAR=bar debian sh -c 'echo $FOO $BAR'
foo bar
```

hostname (`-h`)

```
$ docker run debian hostname
830e47237187
$ docker run -h my-nice-container debian hostname
my-nice-hostname
```

`docker run` — set the container name

`--name` assigns a name for the container
(by default a random name is generated)

```
$ docker run -d -t debian
da005df0d3aca345323e373e1239216434c05d01699b048c5ff277dd691ad535
$ docker run -d -t --name blahblah debian
0bd3cb464ff68eaf9fc43f0241911eb207fefd9c1341a0850e8804b7445ccd21
$ docker ps
CONTAINER ID   IMAGE       COMMAND      CREATED             ..  NAMES
0bd3cb464ff6   debian:7.5  "/bin/bash"  6 seconds ago           blahblah
da005df0d3ac   debian:7.5  "/bin/bash"  About a minute ago      drunk_darwin
$ docker stop blahblah drunk_darwin
```

Note: Names must be unique

```
$ docker run --name blahblah debian true
2015/01/20 19:31:21 Error response from daemon: Conflict, The name blahblah is already assigned
to 0bd3cb464ff6. You have to delete (or rename) that container to be able to assign blahblah to a
 container again.
```

docker run — autoremove

By default the container still exists after command exit

```
$ docker run --name date-ctr debian date
Tue Jan 20 18:38:21 UTC 2015
$ docker start date-ctr
date-ctr
$ docker logs date-ctr
Tue Jan 20 18:38:21 UTC 2015
Tue Jan 20 18:38:29 UTC 2015
$ docker rm date-ctr
date-ctr
$ docker start date-ctr
Error response from daemon: No such container: date-ctr
2015/01/20 19:39:27 Error: failed to start one or more containers
```

With `--rm` the container is automatically removed after exit

```
$ docker run --rm --name date-ctr debian date
Tue Jan 20 18:41:49 UTC 2015
$ docker rm date-ctr
Error response from daemon: No such container: date-ctr
2015/01/20 19:41:53 Error: failed to remove one or more containers
```

Common `rm` idioms

Launch a throwaway container for debugging/testing purpose

```
$ docker run --rm -t -i debian
root@4b71c9a39326:/#
```

Remove all zombie containers

```
$ docker ps -a
CONTAINER ID  IMAGE       COMMAND     CREATED              STATUS
2b291251a415  debian:7.5  "hostname"  About a minute ago   Exited (0) About a mi
6d36a2f07e18  debian:7.5  "false"     2 minutes ago        Exited (1) 2 minutes
0f563f110328  debian:7.5  "true"      2 minutes ago        Exited (0) 2 minutes
4b57d0327a20  debian:7.5  "uname -a"  5 minutes ago        Exited (0) 5 minutes
$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
2b291251a415
6d36a2f07e18
0f563f110328
4b57d0327a20
```

Inspecting the container

command	description
`docker ps`	list running containers
`docker ps -a`	list all containers
`docker logs` [`-f`[7]] *container*	show the container output (*stdout+stderr*)
`docker top` *container* [*ps options*]	list the processes running inside the containers[8]
`docker stats` [*container*]	display live usage statistics[9]
`docker diff` *container*	show the differences with the image (modified files)
`docker port` *container*	list port mappings
`docker inspect` *container...*	show low-level infos (in json format)

[7] with `-f`, `docker logs` follows the output (à la `tail -f`)

[8] `docker top` is the equivalent of the `ps` command in unix

[9] `docker stats` is the equivalent of the `top` command in unix

Interacting with the container

command	description
`docker attach` *container*	attach to a running container (stdin/stdout/stderr)
`docker cp` *container:path hostpath*|- `docker cp` *hostpath*|- *container:path*	copy files from the container copy files into the container
`docker export` *container*	export the content of the container (tar archive)
`docker exec` *container args...*	run a command in an existing container (**useful** for debugging)
`docker wait` *container*	wait until the container terminates and return the exit code
`docker commit` *container image*	commit a new docker image (snapshot of the container)

docker commit example

```
$ docker run --name my-container -t -i debian
root@3b397d383faf:/# echo "echo 'hello!'" >> /etc/bash.bashrc
root@3b397d383faf:/# exit
$ docker start --attach my-container
my-container
hello!
root@3b397d383faf:/# exit
$ docker diff my-container
C /etc
C /etc/bash.bashrc
A /.bash_history
C /tmp
$ docker commit my-container hello
a57e91bc3b0f5f72641f19cab85a7f3f860a1e5e9629439007c39fd76f37c5dd
$ docker rm my-container
my-container
$ docker run --rm -t -i hello
hello!
root@386ed3934b44:/# exit
$ docker history hello
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
ae9924441a13   4 minutes ago   bash                                            2.06kB
c1f0b46af47a   3 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@...  120MB
$ docker image ls
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
debian:latest                        c1f0b46af47a       120MB             0B
hello:latest                         ae9924441a13       120MB             0B
```

Part 3.
Inputs/Outputs

- Persisten data
  - mounted files and directories from the host filesystem
  - named volumes (internal + volume plugins)
- Devices
- Publishing ports (NAT)

`docker run` — external volumes

```
docker run
   --mount type=bind,src=/hostpath,dst=/ctrpath[,OPTIONS]
```

mounts the location */hostpath* from the host filesystem at the location */ctrpath* inside the container

With the "`ro`" option the mount is read-only

With the "`bind-create-src`"[10] option the source directory is automatically created if absent

Purposes:

- store persistent data outside the container
- provide inputs: data, config files (read-only mode)
- inter-process communication (unix sockets, named pipes)

[10] since docker v29.3.0

mount examples (1/2)

Persistent data

```
$ docker run --rm -t -i --mount type=bind,src=/tmp/persistent,dst=/persistent debian
docker: Error response from daemon: invalid mount config for type "bind": bind source path does
not exist: /tmp/persistent
$ docker run --rm -t -i --mount type=bind,src=/tmp/persistent,dst=/persistent,bind-create-src
debian
root@0aeedfeb7bf9:/# echo "blahblah" >/persistent/foo
root@0aeedfeb7bf9:/# exit
$ cat /tmp/persistent/foo
blahblah
$ docker run --rm -t -i --mount type=bind,src=/tmp/persistent,dst=/persistent debian
root@6c8ed008c041:/# cat /persistent/foo
blahblah
```

Inputs (read-only)

```
$ mkdir /tmp/inputs
$ echo hello > /tmp/inputs/bar
$ docker run --rm -t -i --mount type=bind,src=/tmp/inputs,dst=/inputs,ro debian
root@05168a0eb322:/# cat /inputs/bar
hello
root@05168a0eb322:/# touch /inputs/foo
touch: cannot touch `/inputs/foo': Read-only file system
```

mount examples (2/2)

Named pipe

```
$ mkfifo /tmp/fifo
$ docker run -d --mount type=bind,src=/tmp/fifo,dst=/fifo debian sh -c 'echo blah blah> /fifo'
ff0e44c25e10d516ce947eae9168060ee25c2a906f62d63d9c26a154b6415939
$ cat /tmp/fifo
blah blah
```

Unix socket

```
$ docker run --rm -t -i --mount type=bind,src=/dev/log,dst=/dev/log debian
root@56ec518d3d4e:/# logger blah blah blah
root@56ec518d3d4e:/# exit
$ sudo journalctl -e | grep blah
Jun 08 15:21:37 chopper root[116150]: blah blah blah
```

`docker run` — named volumes

Named volumes (`type=volume`)

- stored inside `/var/lib/docker`
- lifecycle managed with the `docker volume` command
- plugin API to provide shared storage over a cluster/cloud[11]

```
$ docker volume create my-volume
my-volume
$ docker volume ls
DRIVER              VOLUME NAME
local               my-volume
$ docker run --rm -t -i --mount type=volume,src=my-volume,dst=/vol busybox
/ # echo foo > /vol/bar
/ # ^D
$ docker volume inspect my-volume | grep Mountpoint
        "Mountpoint": "/var/lib/docker/volumes/my-volume/_data",
$ docker run --rm -t -i --mount src=my-volume,dst=/vol busybox cat /vol/bar
foo
$ docker volume rm my-volume
my-volume
```

[11] https://docs.docker.com/engine/tutorials/dockervolumes/

initialisation: bind volumes vs named volumes

https://docs.docker.com/engine/storage/

- bind volumes are created empty
- named volumes are created with a copy of the image content at the same mount point

```
$ docker run --rm -t alpine ls /etc/apk
arch              keys              protected_paths.d  repositories      world

$ docker run --rm -t --mount /tmp/dummy:/etc/apk alpine ls /etc/apk
$ ls /tmp/dummy/
$

$ docker run --rm -t --volume dummy:/etc/apk alpine ls /etc/apk
arch              keys              protected_paths.d  repositories      world
$ ls /var/lib/docker/volumes/dummy/_data
arch              keys              protected_paths.d  repositories      world
```

the old -v/--volume option

Volumes can also be mounted using the `-v` deprecated[12] option

Bind volumes:

```
docker run -v /hostpath:/ctrpath[:ro]
```

is equivalent to

```
docker run --mount type=bind,src=/hostpath,dst=/ctrpath,bind-create-src[,ro]
```

Named volumes:

```
docker run -v volume:/ctrpath[:ro]
```

is equivalent to

```
docker run --mount type=volume,src=volume,dst=/ctrpath[,ro]
```

and

```
docker run --mount src=volume,dst=/ctrpath[,ro]
```

https://docs.docker.com/engine/storage/volumes/#syntax

[12] but still widely used

`docker run` — grant access to a device

By default devices are not usable inside the container

```
$ docker run --rm debian  fdisk -l /dev/sda
fdisk: cannot open /dev/sda: No such file or directory

$ docker run --rm debian  sh -c 'mknod /dev/sda b 8 0 && fdisk -l /dev/sda'
fdisk: cannot open /dev/sda: Operation not permitted

$ docker run --rm --mount /dev/sda:/dev/sda debian  fdisk -l /dev/sda
fdisk: cannot open /dev/sda: Operation not permitted
```

They can be whitelisted with `--device`

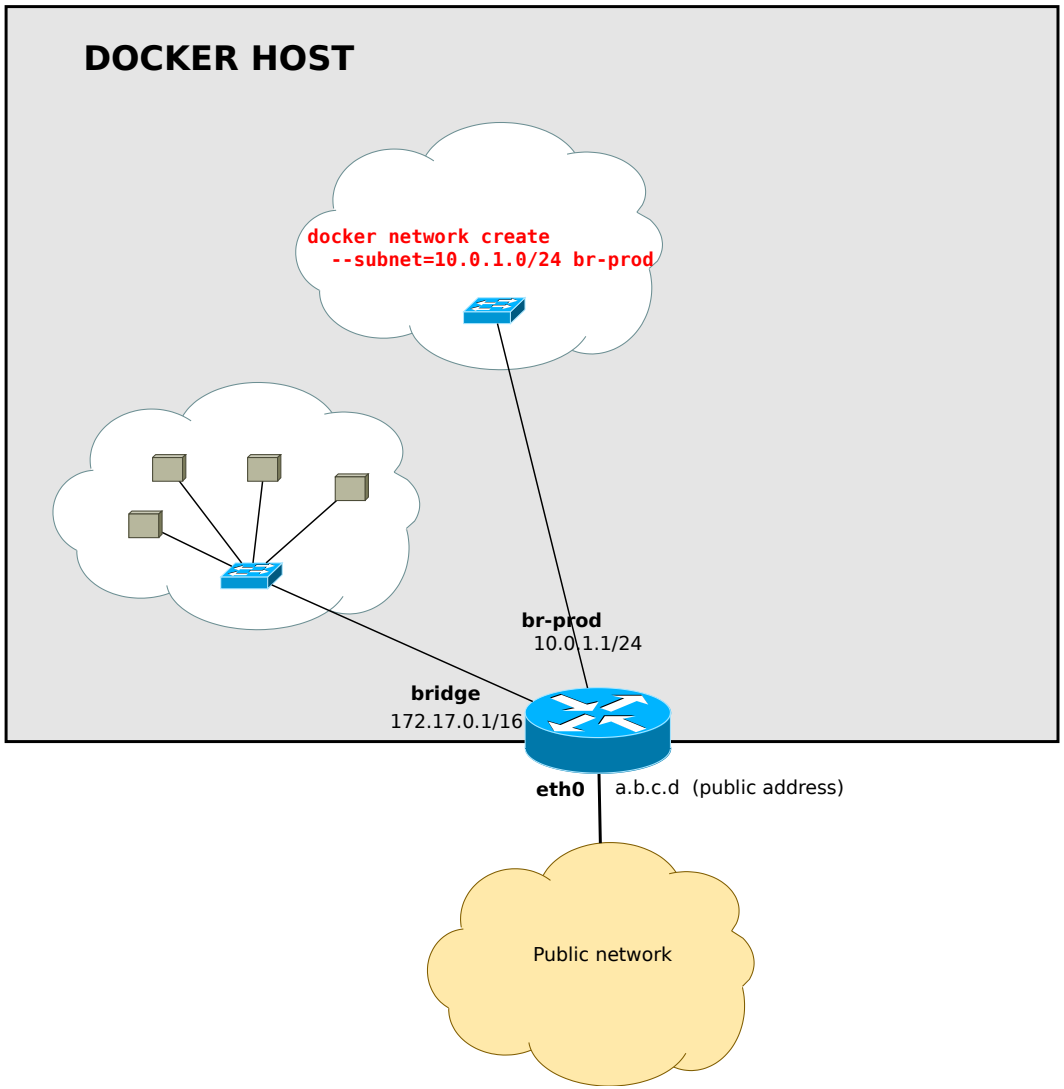
`docker run --device` *`/hostpath`*`[:`*`/containerpath`*` ] ...`

```
$ docker run --rm --device /dev/sda debian  fdisk -l /dev/sda
Disk /dev/sda: 250.1 GB, 250059350016 bytes
...
```

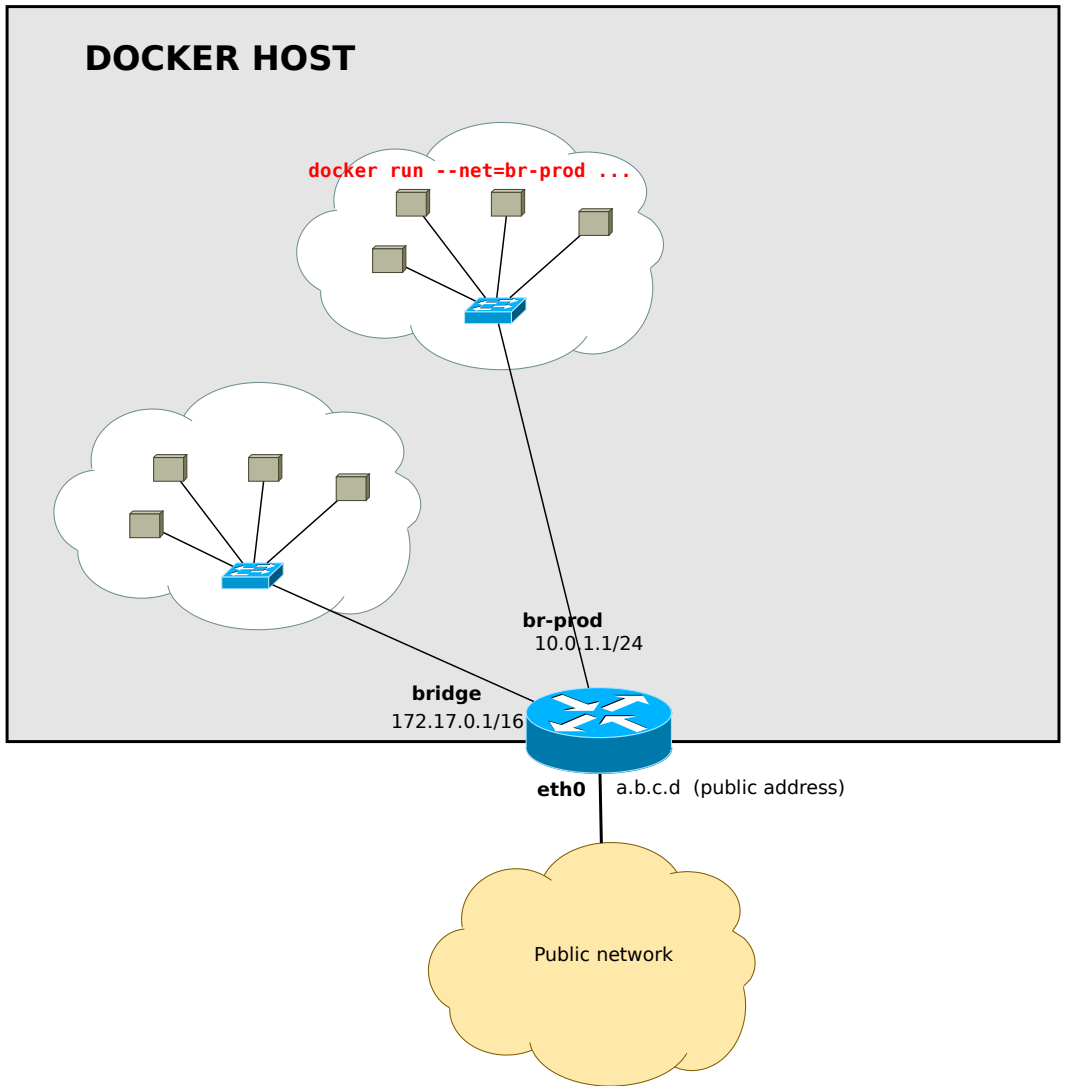
User-defined networks (since v1.9.0)

- by default new containers are connected to the main network (named “bridge”, 172.17.0.0/16)
- the user can create additional networks:
  `docker network create` *NETWORK*
- newly created containers are connected to one network:
  `docker run --net=`*NETWORK*
- container may be dynamically attached/detached to any network:
  `docker network connect` *NETWORK CONTAINER*
  `docker network disconnect` *NETWORK CONTAINER*
- networks are isolated from each other, communications is possible by attaching a container to multiple networks

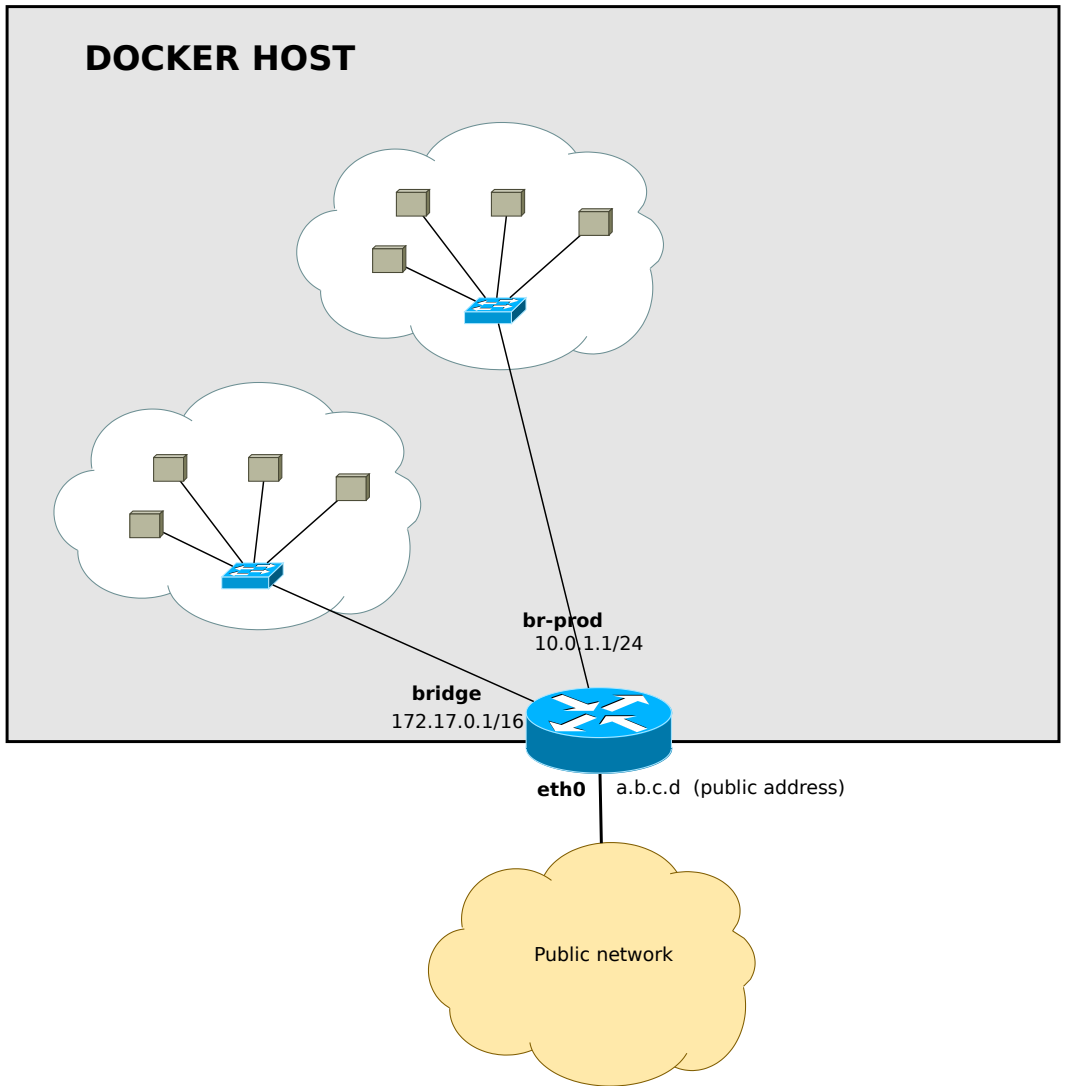
User-defined networks example



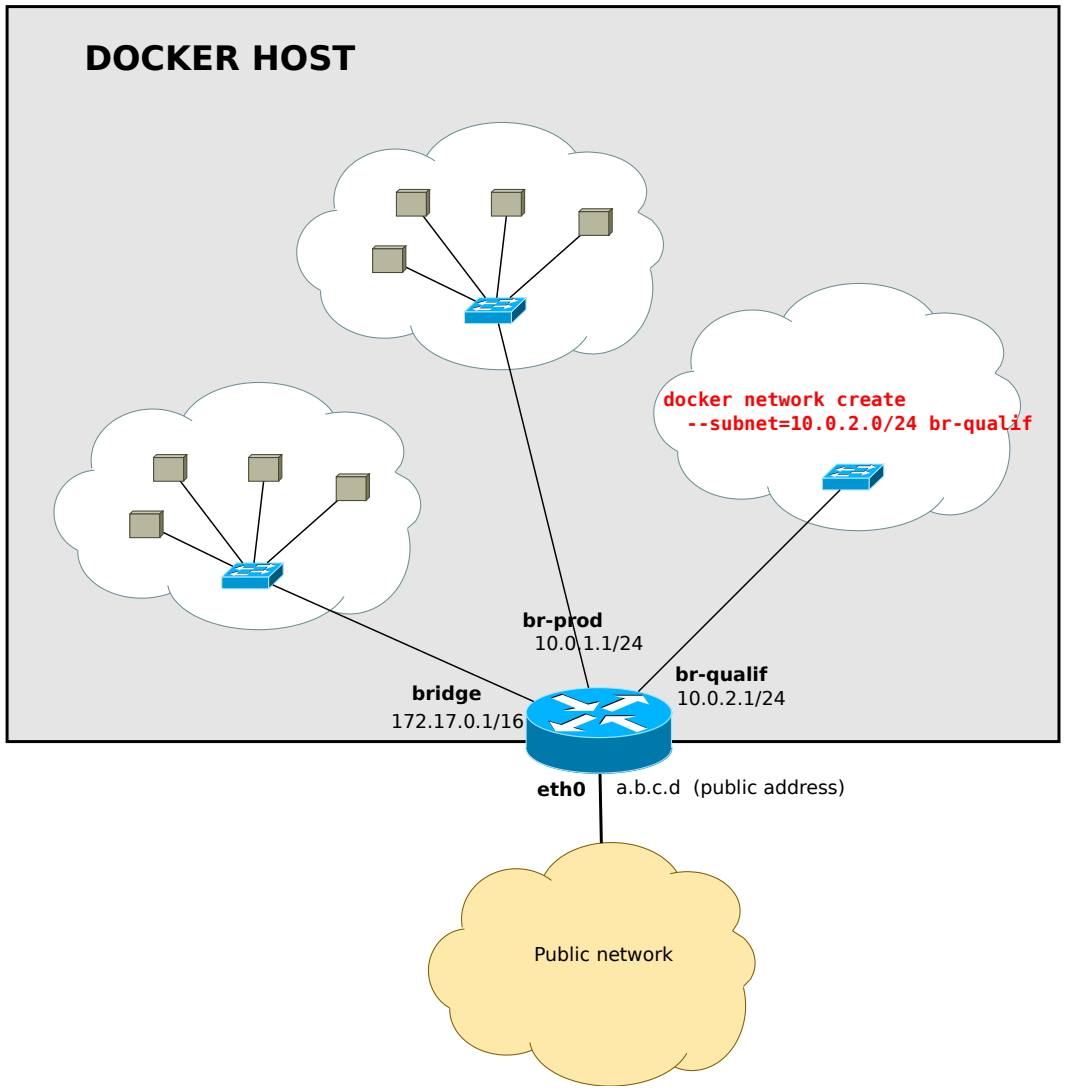
User-defined networks example



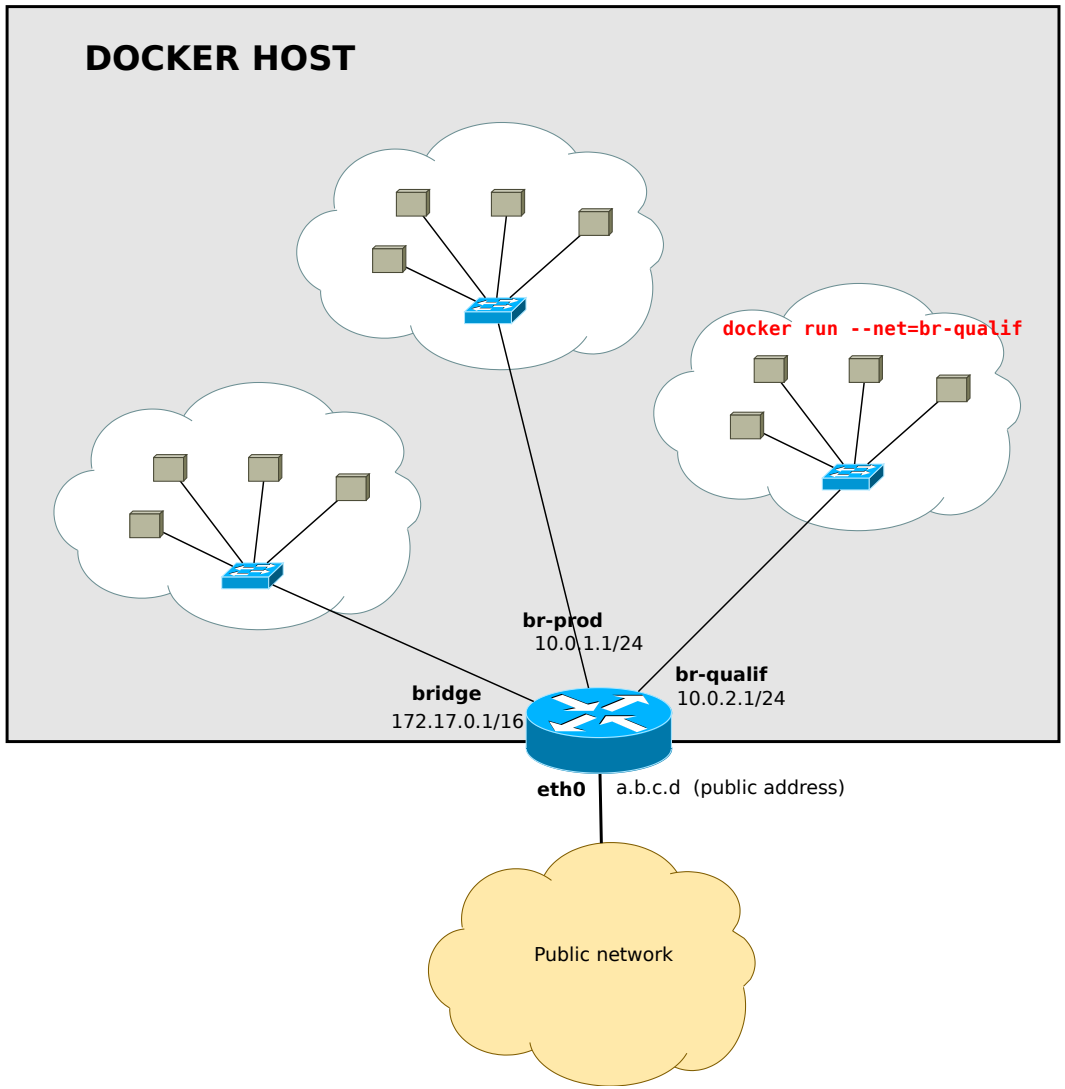
User-defined networks example



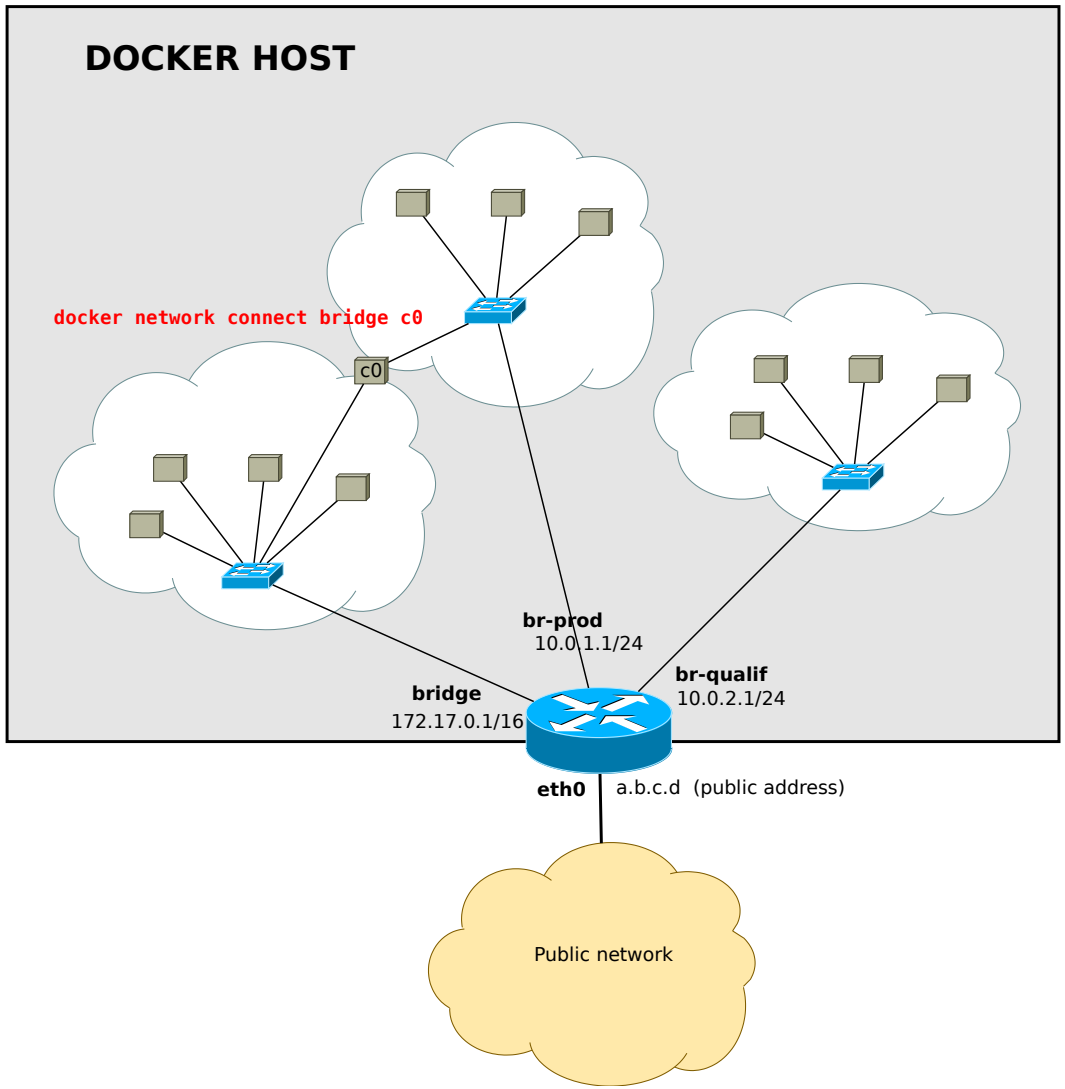
User-defined networks example



User-defined networks example



User-defined networks example



`docker run` — publish a TCP port

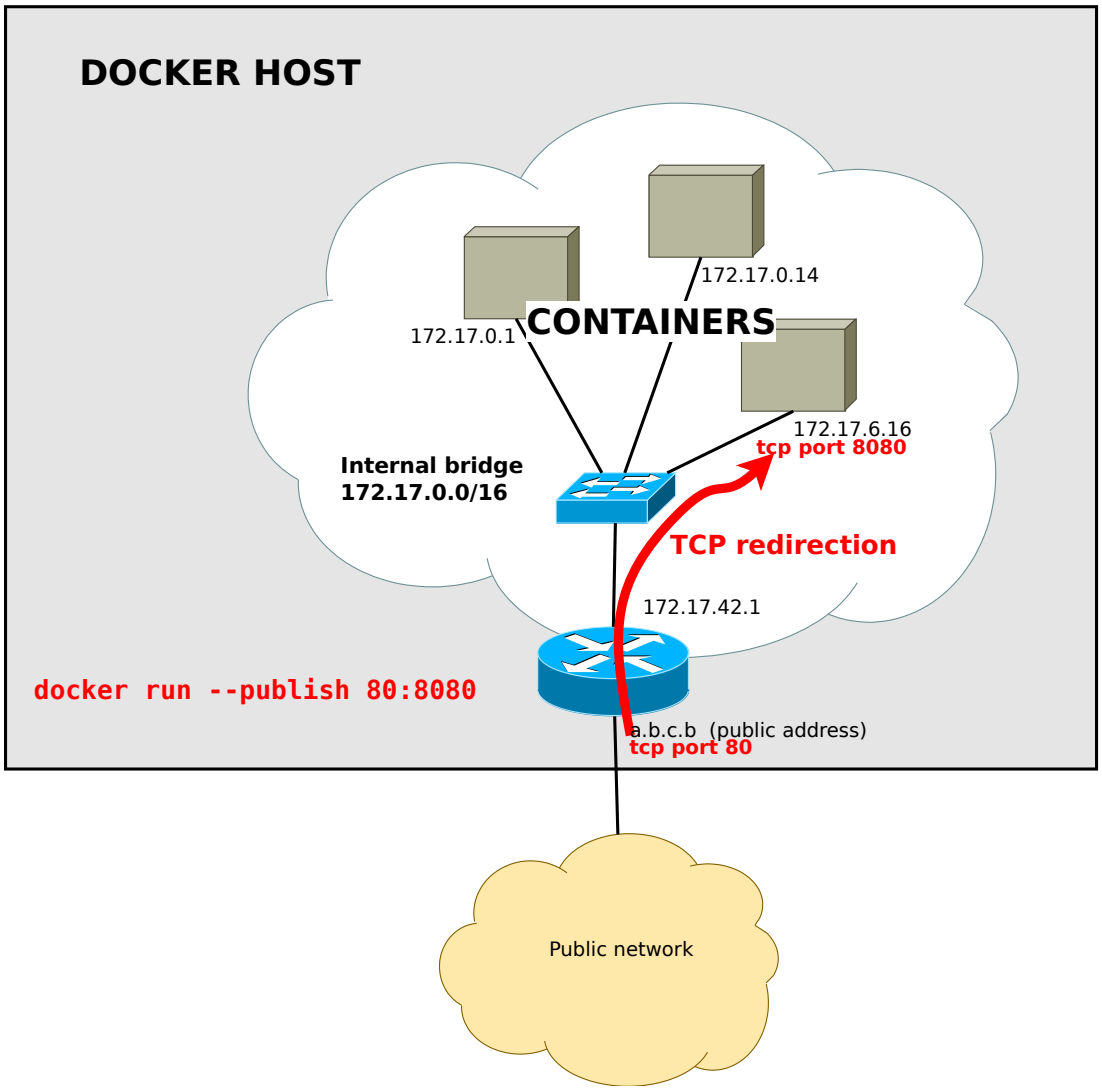
Containers are deployed in a private network, they are not reachable from the outside (unless a redirection is set up)

```
docker run -p [ipaddr:]hostport:containerport
```

→ redirect incoming connections to the TCP port *hostport* of the host to the TCP port *containerport* of the container

The listening socket binds to 0.0.0.0 (all interfaces) by default or to *ipaddr* if given

publish example



publish example

bind to all host addresses

```
$ docker run -d -p 80:80 nginx
52c9105e1520980d49ed00ecf5f0ca694d177d77ac9d003b9c0b840db9a70d62

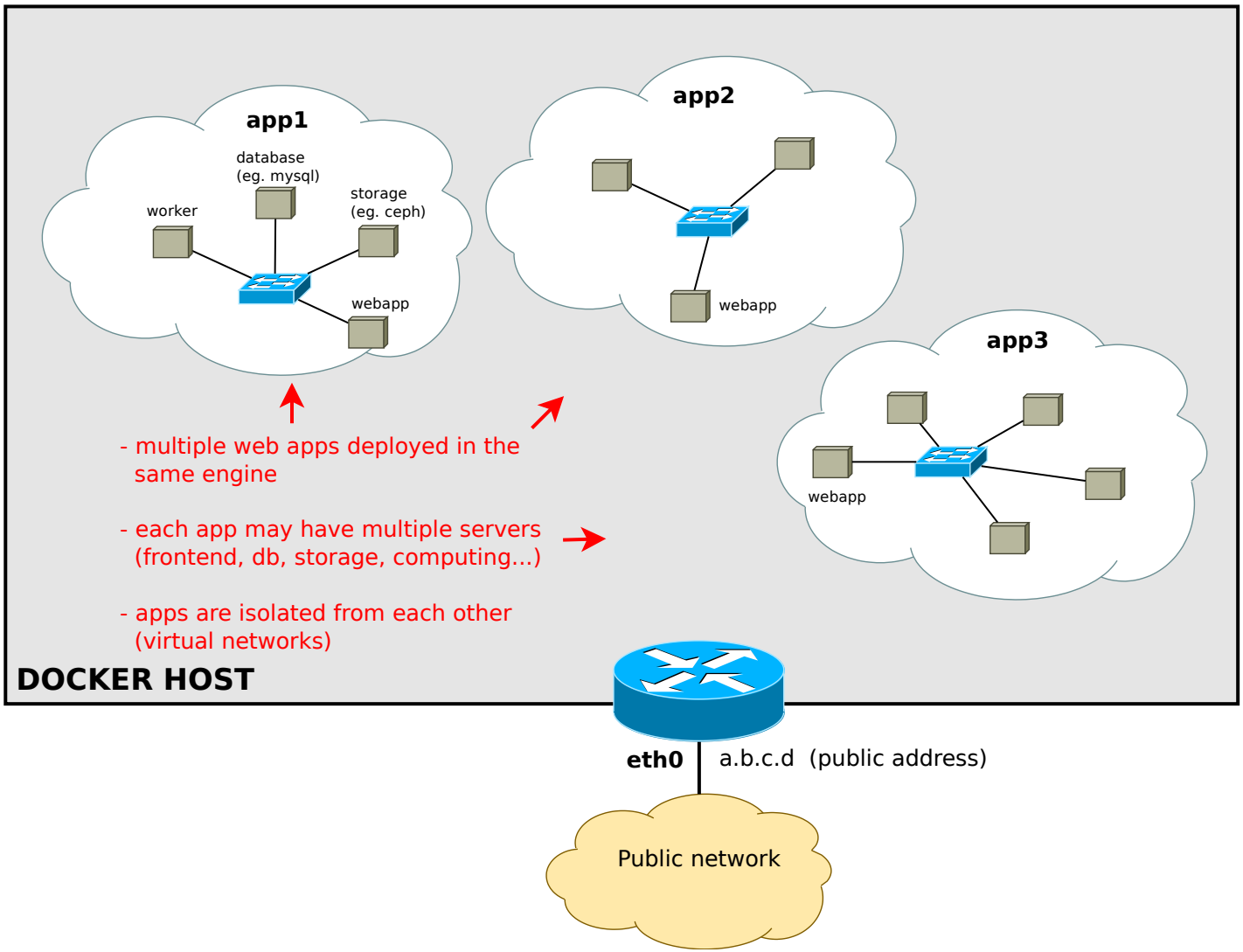
$ wget -nv http://localhost/
2016-01-12 18:32:52 URL:http://localhost/ [612/612] -> "index.html" [1]
$ wget -nv http://172.17.42.1/
2016-01-12 18:33:14 URL:http://172.17.42.1/ [612/612] -> "index.html" [1]
```

bind to 127.0.0.1

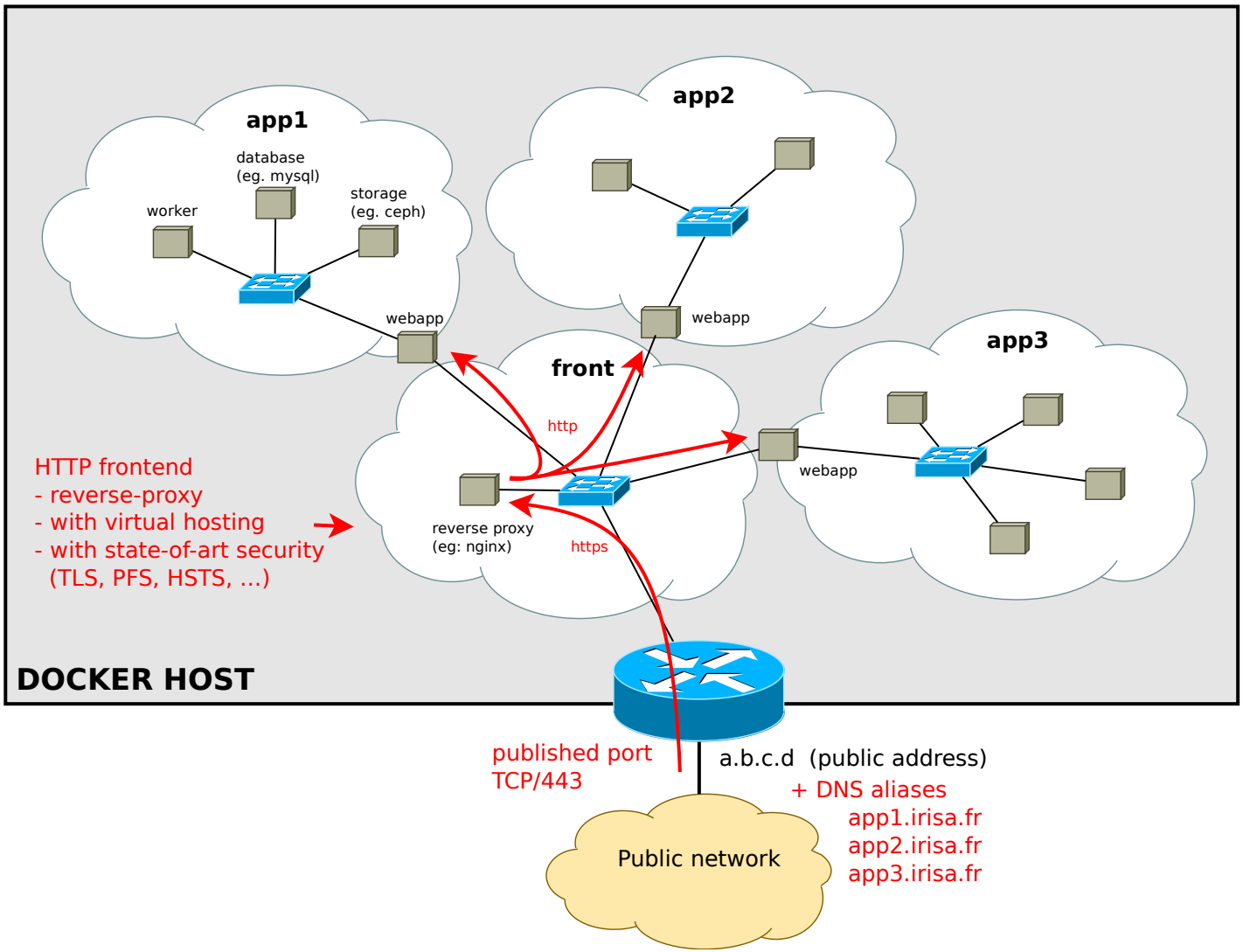
```
$ docker run -d -p 127.0.0.1:80:80 nginx
4541b43313b51d50c4dc2722e741df6364c5ff50ab81b828456ca55c829e732c

$ wget -nv http://localhost/
2016-01-12 18:37:10 URL:http://localhost/ [612/612] -> "index.html.1" [1]
$ wget http://172.17.42.1/
--2016-01-12 18:38:32--  http://172.17.42.1/
Connecting to 172.17.42.1:80... failed: Connection refused.
```

The whole picture



The whole picture



Part 4.
Managing docker images

Docker images

A docker image is a snapshot of the filesystem + some metadata

- immutable
- copy-on-write storage
  - for instantiating containers
  - for creating new versions of the image (multiple layers)
- identified by unique hex IDs
  - `Image ID`: randomly genreated
  - `Digest`: hashed from the content
- may be tagged[13] with a human-friendly name
  eg: `debian:wheezy` `debian:jessie` `debian:latest`

[13] possibly multiple times

Image management commands

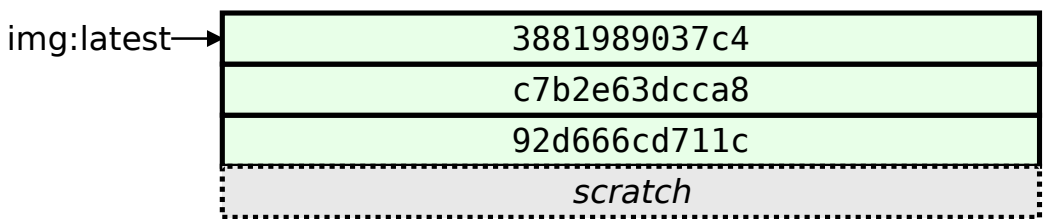
command	description
`docker images`	list all local images
`docker history` *image*	show the image history (list of ancestors)
`docker inspect` *image...*	show low-level infos (in json format)
`docker tag` *image tag*	tag an image
`docker commit` *container image*	create an image (from a container)
`docker import` *url*|- [*tag*]	create an image (from a tarball)
`docker rmi` *image...*	delete images

Example: images & containers



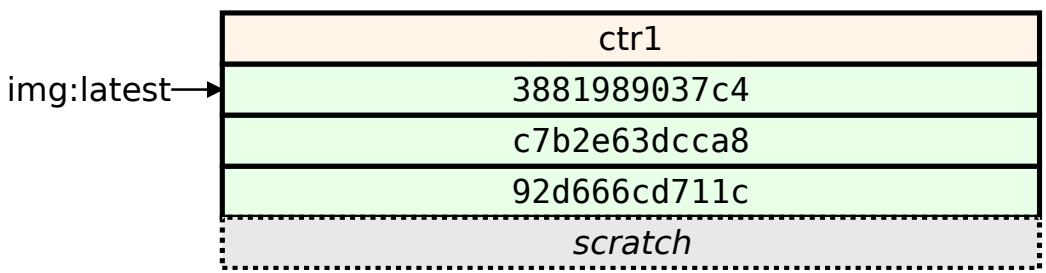
Example: images & containers

```
docker pull img
```



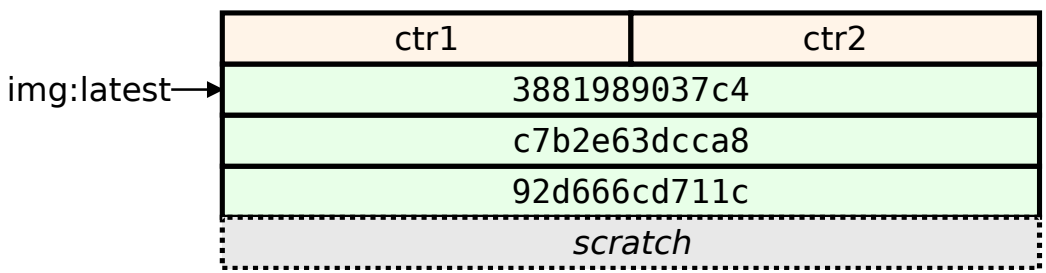
Example: images & containers

```
docker run --name ctr1 img
```



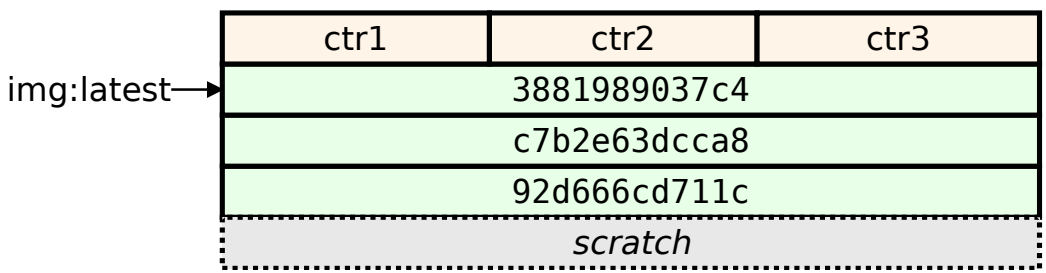
Example: images & containers

```
docker run --name ctr2 img
```



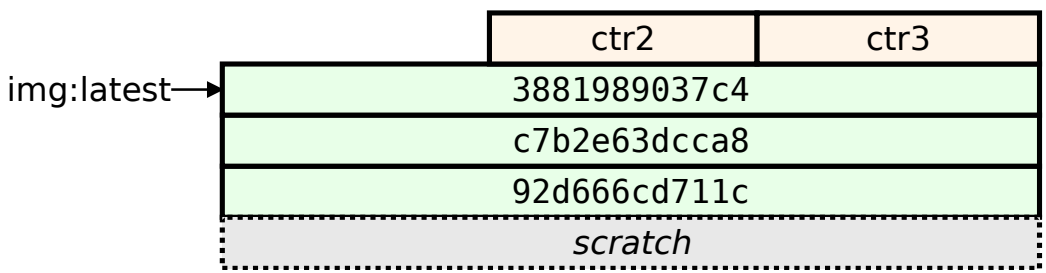
Example: images & containers

```
docker run --name ctr3 img
```



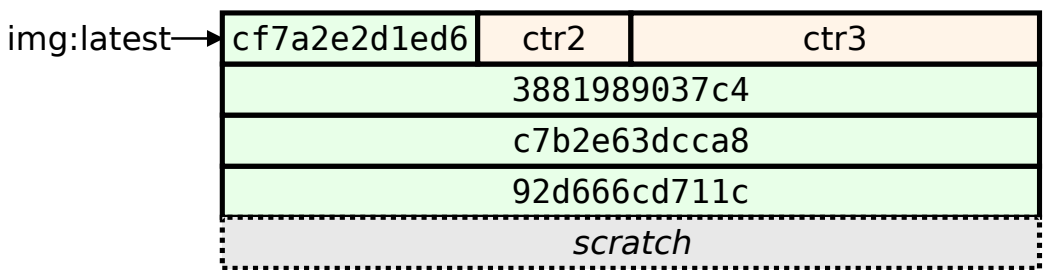
Example: images & containers

```
docker rm ctr1
```



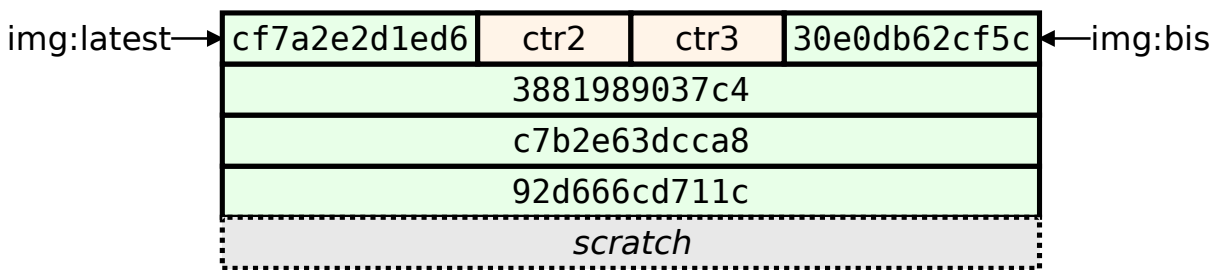
Example: images & containers

```
docker commit ctr2 img
```



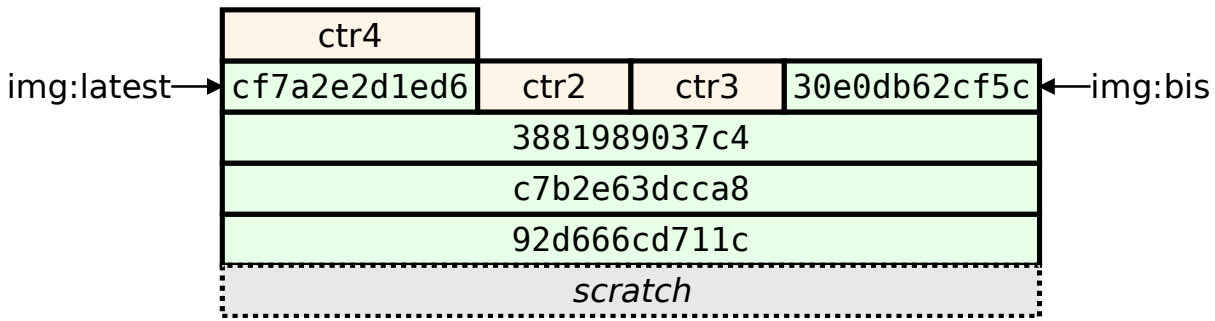
Example: images & containers

```
docker commit ctr3 img:bis
```



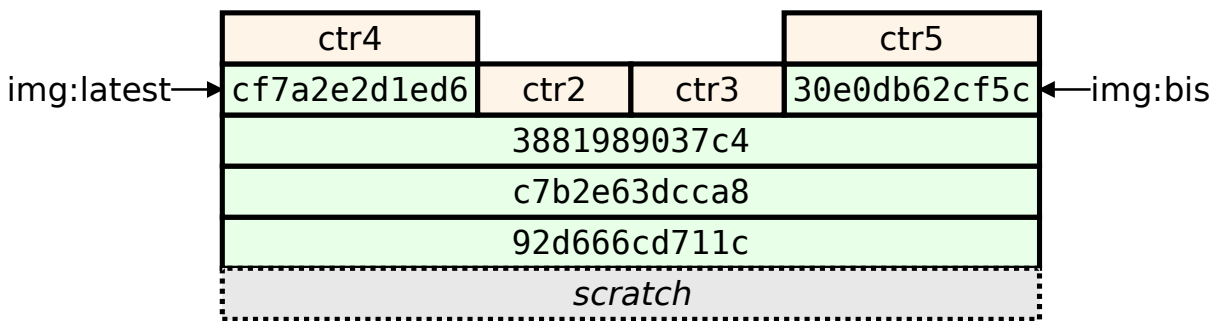
Example: images & containers

```
docker run --name ctr4 img
```



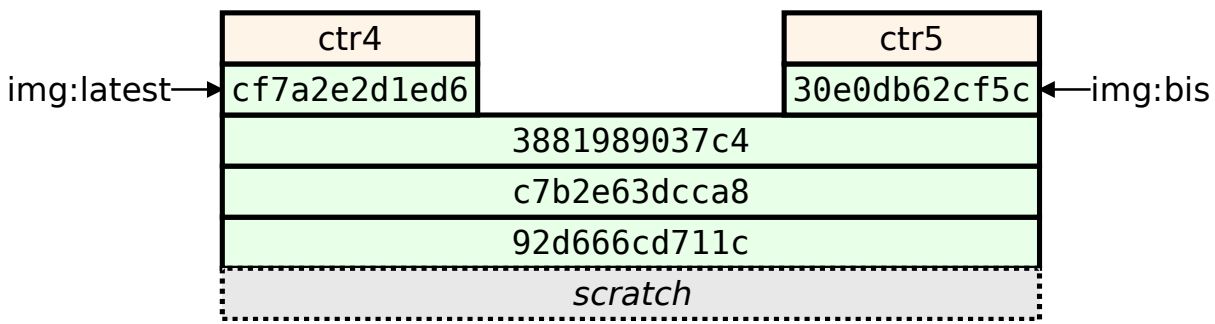
Example: images & containers

```
docker run --name ctr5 img:bis
```



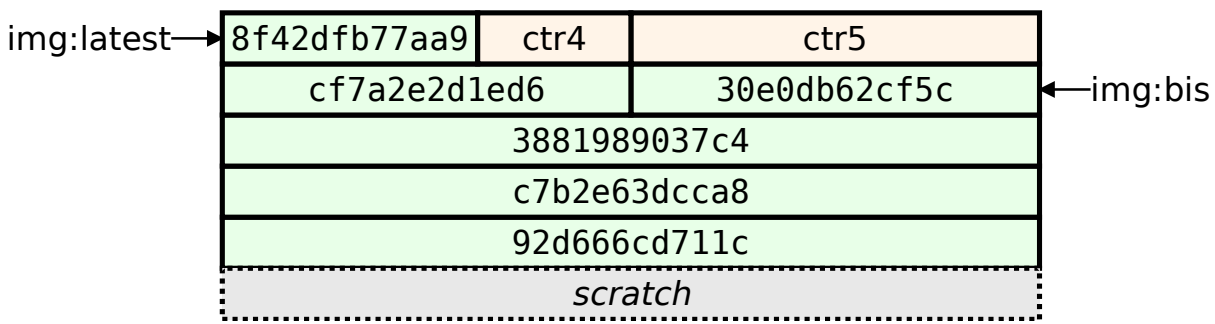
Example: images & containers

```
docker rm ctr2 ctr3
```



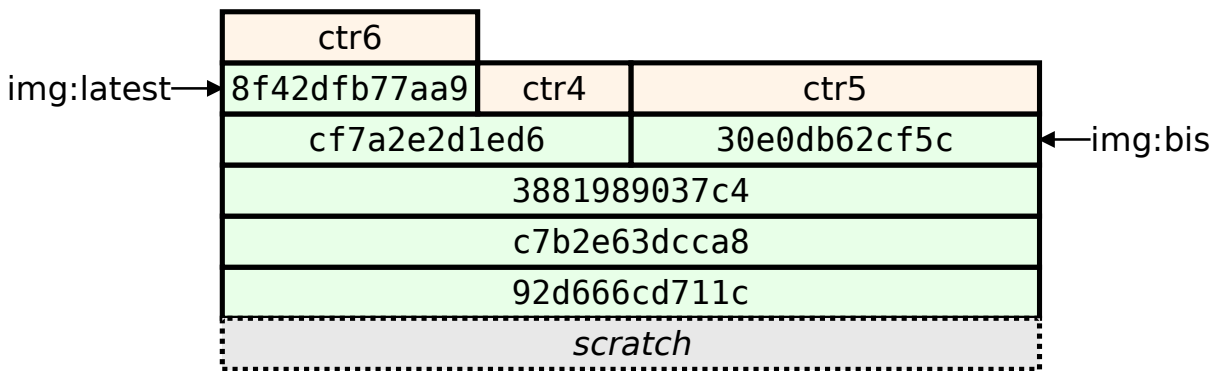
Example: images & containers

```
docker commit ctr4 img
```



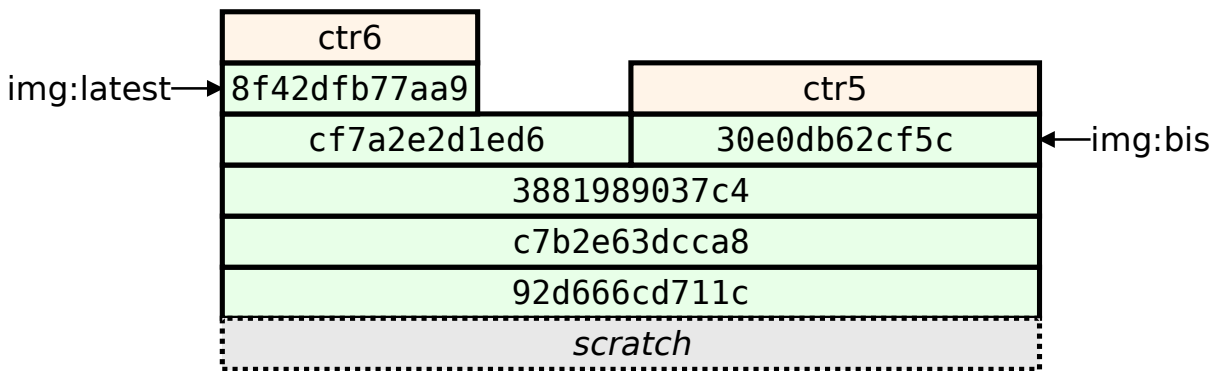
Example: images & containers

```
docker run --name ctr6 img
```



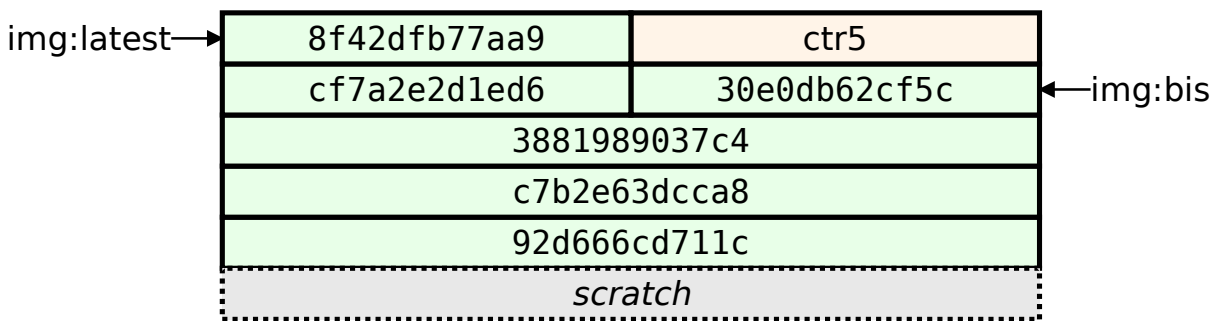
Example: images & containers

```
docker rm ctr4
```



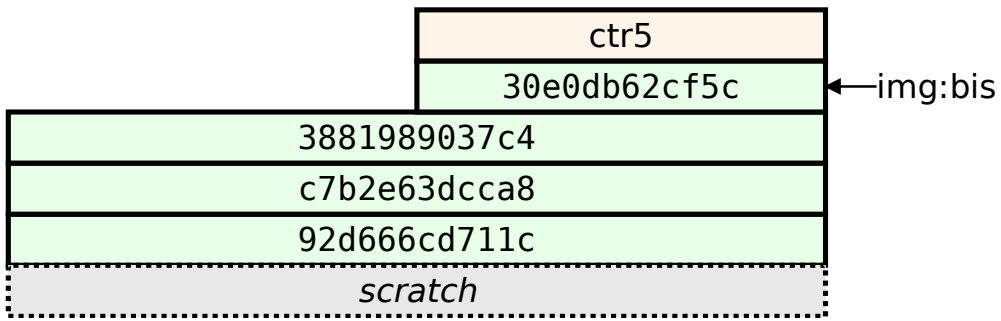
Example: images & containers

```
docker rm ctr6
```



Example: images & containers

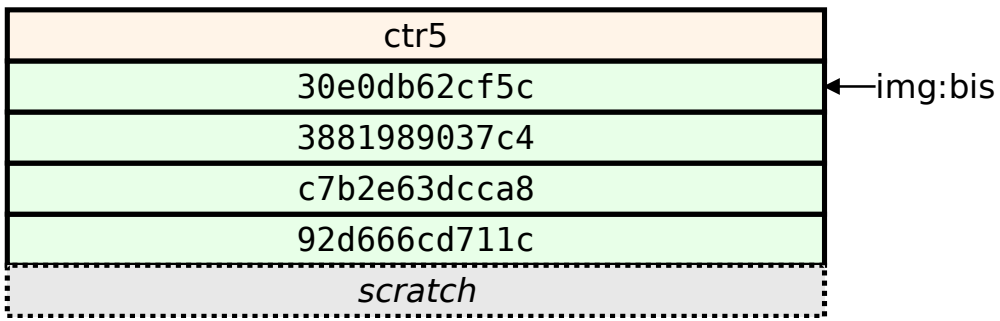
```
docker rmi img
```



Example: images & containers

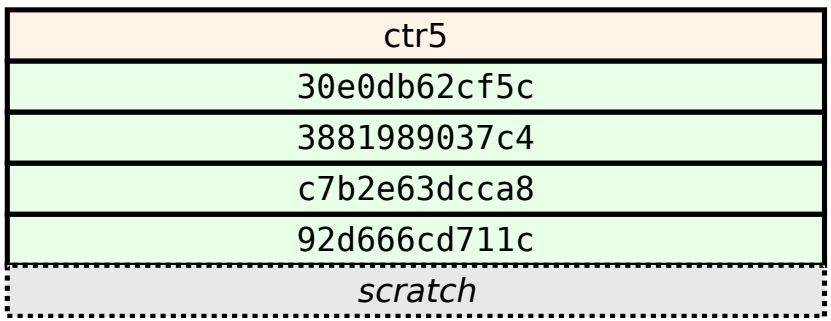
```
docker rmi img:bis
```

Error: image img:bis is reference by ctr5



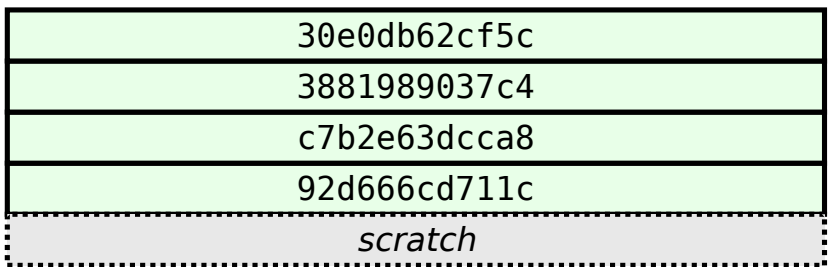
Example: images & containers

```
docker rmi -f img:bis
```



Example: images & containers

```
docker rm ctr5
```



Example: images & containers

```
docker rmi 30e0
```



Images vs. Layers

since docker v1.10
layers are hidden to the user
(implementation detail)

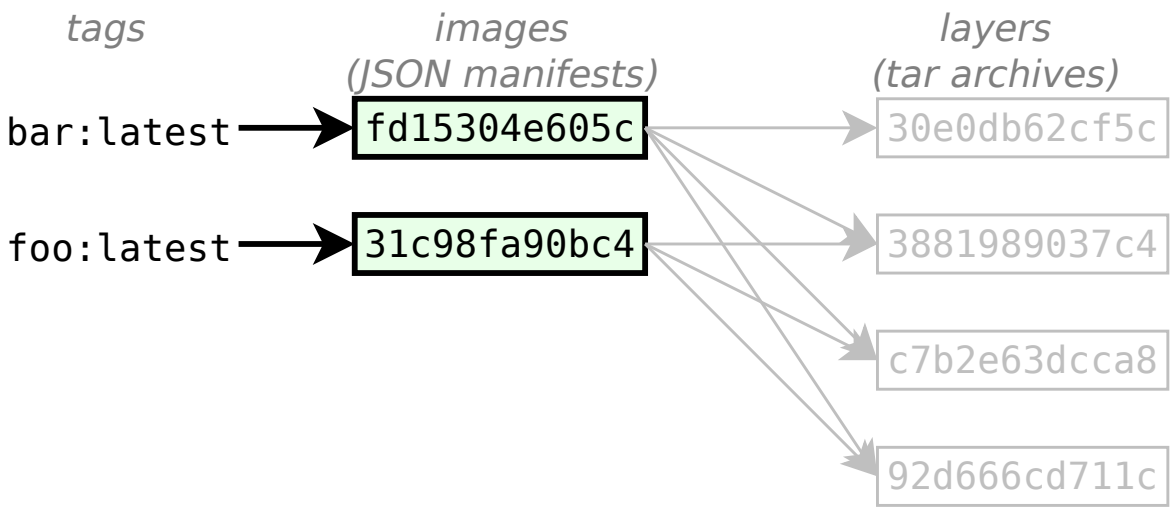


Image tags

A docker tag is made of two parts: *"REPOSITORY*: *TAG"*

The *TAG* part identifies the version of the image. If not provided, the default is "`:latest`"

```
$ docker images
REPOSITORY  TAG           IMAGE ID      CREATED        VIRTUAL SIZE
debian      8             835c4d274060  2 weeks ago    122.6 MB
debian      8.0           835c4d274060  2 weeks ago    122.6 MB
debian      jessie        835c4d274060  2 weeks ago    122.6 MB
debian      rc-buggy      350a74df81b1  7 months ago   159.9 MB
debian      experimental  36d6c9c7df4c  7 months ago   159.9 MB
debian      6.0.9         3b36e4176538  7 months ago   112.4 MB
debian      squeeze       3b36e4176538  7 months ago   112.4 MB
debian      wheezy        667250f9a437  7 months ago   115 MB
debian      latest        667250f9a437  7 months ago   115 MB
debian      7.5           667250f9a437  7 months ago   115 MB
debian      unstable      24a4621560e4  7 months ago   123.6 MB
debian      testing       7f5d8ca9fdcf  7 months ago   121.8 MB
debian      stable        caa04aa09d69  7 months ago   115 MB
debian      sid           f3d4759f77a7  7 months ago   123.6 MB
debian      7.4           e565fbbc6033  9 months ago   115 MB
debian      7.3           b5fe16f2ccba  11 months ago  117.8 MB
```

Tagging conventions (1/2)

Local tags may have arbitrary names, however the `docker push` and `docker pull` commands expect some conventions

The *REPOSITORY* identifies the origin of the image, it may be:

- a name (eg: `debian`)
  → refers to a repository on the official registry
  → `https://hub.docker.com/`

- a hostname+name (eg: `some.server.com/repo`)
  → refers to an arbitrary server supporting the registry API
  → `https://docs.docker.com/reference/api/registry_api/`

Tagging conventions (2/2)

Use slashes to delimit namespaces (for subprojects):

image name	**description**
`debian`	(semi-)official debian images
`fedora`	official fedora images
`fedora/apache`	apache images provided by the fedora project
`fedora/couchdb`	couchdb images provided by the fedora project

Image transfer commands

Using the registry API

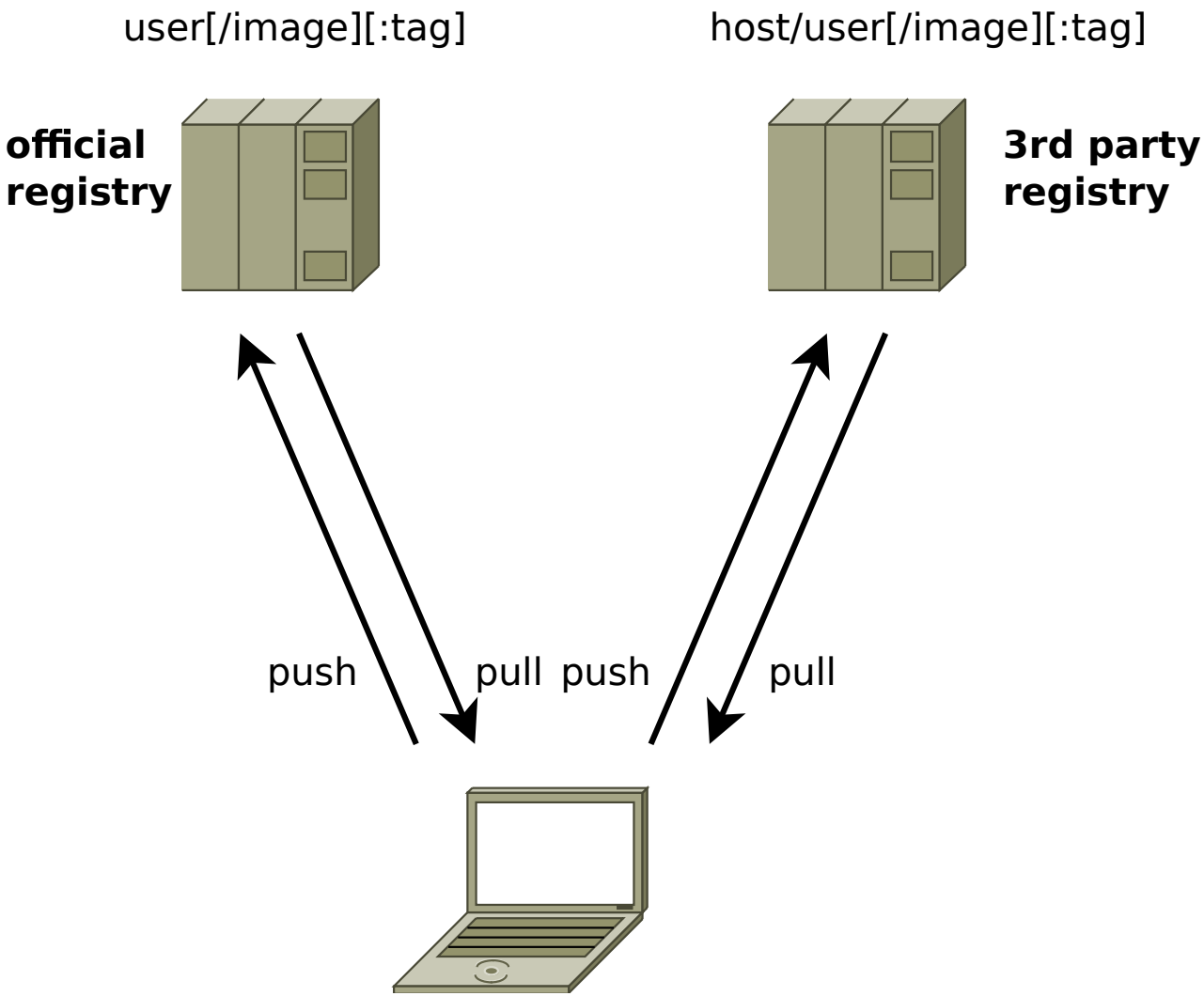
`docker pull` *repo*[:*tag*]...	pull an image/repo from a registry
`docker push` *repo*[:*tag*]...	push an image/repo from a registry
`docker search` *text*	search an image on the official registry
`docker login` ...	login to a registry
`docker logout` ...	logout from a registry

Manual transfer

`docker save` *repo*[:*tag*]...	export an image/repo as a tarball
`docker load`	load images from a tarball
`docker-ssh`[14] ...	proposed script to transfer images between two daemons over ssh

[14] `https://github.com/a-ba/docker-utils/`

Transferring images



Part 5.
Docker builder

What is the Docker builder ?

Docker's builder relies on

- a DSL describing how to build an image
- a cache for storing previous builds and have quick iterations

The builder input is a **context**, i.e. a directory containing:

- a file named `Dockerfile` which describes how to build the image
- arbitrary files to be copied into the image during the build
- an optional `.dockerignore` file that lists patterns of filenames that must be ignored by the builder[15]

As of docker 23 the builder is based on `buildkit`, which can be run as a standalone tool when docker is not available (eg: in a CI job)

[15] similar to the `.gitignore` file for `git`

Build an image

```
docker build [ -t tag ] context
```

- *context* is a directory containing the source files
- *tag* is an optional tag to be applied to the generated image

- The builder executes the commands listed in *context*/`Dockerfile`
- Each command creates a new layer
- At the end of the build the layers are grouped in a new image (optionally tagged as *tag*)

Dockerfile example

```
# base image: last debian release
FROM debian:latest

# install nginx with apt (+apply all upgrades)
#
# NOTES:
#  - the "--mount=type=cache" tells the builder that the content of the /var/cache/apt/
#    directory can be reused for future builds of the same image
#    (so as to speed up the build: apt will not have to download the packages again)
#  - a "\" at end of line is used to let the command span over multiple lines
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    apt-get -qqy update       &&\
    apt-get -qqy dist-upgrade &&\
    apt-get -qqy install nginx

# set the default container command
# -> this runs nginx in the foreground
CMD ["nginx", "-g", "daemon off;"]

# Tell the docker engine that there will be somenthing listening on the tcp port 80
EXPOSE 80
```

`Dockerfile` format

https://docs.docker.com/reference/builder/

- comments start with "#"
- commands fit on a single line
  (possibly continued with \)
- first command must be a `FROM`
  (indicates the parent image or `scratch` to start from scratch)

Builder instructions (1/3)

Instructions affecting the image filesystem

instruction	description
`FROM` *image*|`scratch`	base image for the build
`COPY` *path dst*	copy *path* from the context[16] into the container at location *dst*
`ADD` *src dst*	same as `COPY` but untar archives and accepts http urls
`RUN` *command*	run an arbitrary command inside the container

Note: commands may be expressed as a list (exec) or a string (shell)

```
# exec form
RUN ["apt-get", "update"]

# shell form
RUN apt-get update       # equivalent to: RUN ["/bin/sh", "-c", "apt-get update"]
```

[16] excluding patterns listed in `.dockerignore`

Builder instructions (2/3)

Instructions setting the default container config[18]

instruction	description
`CMD` *command* `ENTRYPOINT` *command*	command run inside the container entrypoint[17]
`USER` *name*[:*group*] `WORKDIR` *path* `ENV` *name*="*value*"...	user running the command working directory environment variables
`STOPSIGNAL` *signal*	signal to be sent to terminate the container*(instead of SIGTERM)*
`HEALTHCHECK CMD` *command*	test command to check whether the container works well
`EXPOSE` *port*... `VOLUME` *path*...	listened TCP/UDP ports mount-point for external volumes
`LABEL` *name*="*value*"...	arbitrary metadata

[17] the `ENTRYPOINT` is a commmand that wraps the `CMD` command

[18] i.e. the default configuration of containers running this image

Builder instructions (3/3)

Extra instructions

instruction	description
`ARG` *name[=value]*	build-time variables

- build-time variables are usable anywhere in the Dockerfile (*by variable expansion:* `$VARNAME`) and are tunable at build time: "`docker build --build-arg` *name=value ...*"

Builder cache

Each layer created by the builder is fingerprinted according to:

- the ID of the previous image
- the command and its arguments
- the content of the imported files (for `ADD` and `COPY`)

⚠ `RUN`'s side-effects are not fingerprinted

When rebuilding an image docker will reuse a previous image if its fingerprint is the same

Good practices[20] for docker files

- use a base image that is stable and actively maintained
  (eg. `debian:latest`)
- run the app as PID 1 inside the container (to be killable)
  → write `CMD ["app", "arg"]` instead of `CMD app arg`[19]
- standardise the config, but allow the admin to override it with env variables or additional config files
  (eg. `ENV MYSQL_HOST="mysql"`)

[19] which would be expanded to `CMD ["/bin/sh", "-c", "app arg"]` and runs the command as a subprocess of `/bin/sh`

[20] see also `https://docs.docker.com/engine/userguide/eng-image/dockerfile_best-practices/`

Multi-stage build

- a `Dockerfile` may build multiple images
- each `FROM` line marks the beginning of a new image with an optional name: `FROM` *base* *[*`AS` *name]*
- named images may be referenced by other images as:
  - a base image, eg: `FROM` *previous_stage*
  - a content source, eg: `COPY --from=`*previous_stage* *...*
- by default `docker build` builds the last image[21] but a different target may be selected with `--target=`*name*
- multiple targets may be built concurrently with: `docker buildx bake`
- may be used for: building binaries, providing mutiple variant of an image (production, development, ...), running tests, ...

[21] and its dependencies

Multi-stage build example

```
#======= Stage 1: build the app from sources =======#
FROM debian AS builder
# update the package lists an install the build dependencies
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    apt-get -qqy update && apt-get -qqy install gcc make libacme-dev

# install the sources in /opt/src and build the application
COPY . /opt/src
RUN cd /opt/src && ./configure && make

# install the files into an arbitrary root (here: /tmp/dst)
# (this creates /tmp/dst/usr/local/bin/myapp and possibly other files)
RUN cd /opt/src && make install DESTDIR=/tmp/dst

#======= Stage 2: final image ================#
FROM debian
# update the package lists and install the runtime dependencies
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    apt-get -qqy update && apt-get -qqy install libacme1.0

# install the app built in stage 1
COPY --from=builder /opt/dst/. /

CMD ["myapp"]
```

Part 6.
Security

- host/container isolation
- container/container isolation
- other security considerations

`https://docs.docker.com/engine/security/`

Security strategies

Docker containers are not fully sandboxed from the host machine. They talk with the **same kernel**. You may want to consider strategies to reduce the risks of privilege escalation.

Container/Host isolation

- run the container as an ordinary user (`docker run -u`)
- disable privilege escalation (*no-new-privileges*)
- reduce root privileges (*capabilities*, *seccomp*, *apparmor*)
- run the docker engine in rootless mode
- run the docker engine inside a VM

Container/Container isolation

- disable intercontainer communications (`--icc=false`)
- isolate containers in different networks

Running containers as an ordinary user (1/2)

should be safe, but... images may contain setuid executables

```
$ docker build -t setuid-example - <<EOF
FROM alpine
RUN adduser -D dummy-user
RUN apk add doas && echo permit nopass dummy-user  > /etc/doas.d/dummy.conf
USER dummy-user
EOF
$ docker run --rm -t -i setuid-example
/ $ id
uid=1000(dummy-user) gid=1000(dummy-user) groups=1000(dummy-user)
/ $ doas sh
/ # id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),...
```

→ **deny privilege escalation** (`--security-opt no-new-privileges=true`)

```
$ docker run --rm -t -i --security-opt no-new-privileges=true setuid-example
/ $ doas sh
doas: not installed setuid
/ $
```

Running containers as an ordinary user (2/2)

should be safe, but... uid/gid resolution relies on the image's user db

```
$ docker build -t passwd-example - <<EOF
FROM alpine
RUN echo dummy-user:x:0:0::/root:/bin/sh >> /etc/passwd
EOF
$ docker run --rm -t -i --security-opt no-new-privileges=true -u dummy-user passwd-example
/ # id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),...
```

→ **run the container with numeric uid/gid** (`--user` *uid*:*gid*)

```
$ docker run --rm -t -i --security-opt no-new-privileges=true -u 1000:1000 passwd-example
~ $ id
uid=1000 gid=1000 groups=1000
```

Reduced root capabilities

- kernel capabilities supported since docker v1.2
- containers use a default set limited to 14 capabilities[22]:

AUDIT_WRITE	CHOWN	NET_RAW	SETPCAP
DAC_OVERRIDE	FSETID	SETGID	KILL
NET_BIND_SERVICE	FOWNER	SETUID	
SYS_CHROOT	MKNOD	SETFCAP	

- add additional capabilities: `docker run --cap-add=`*XXXXX* ...
- drop unnecessary capabilities: `docker run --cap-drop=`*XXXXX* ...
  → should use `--cap-drop=all` for most containers

```
$ docker run --rm -t -i debian
root@04223cbb1334:/# ip addr replace 172.17.0.42/16 dev eth0
RTNETLINK answers: Operation not permitted
root@04223cbb1334:/# exit

$ docker run --rm -t -i --cap-add NET_ADMIN debian
root@9bf2a570a6a6:/# ip addr replace 172.17.0.42/16 dev eth0
root@9bf2a570a6a6:/#
```

[22] over the 38 capabilities defined in the kernel (man 7 capabilities)

Reduced syscall whitelist

seccomp-bpf == fine-grained acces control to kernel syscalls

- enabled by default since docker v1.10
- default built-in profile[23] whitelists only harmless syscalls[24]
- alternative configs:
  - disable seccomp (`--security-opt=seccomp:unconfined`)
  - provide a customised profile (derived from the default[25])

```
$ docker run --rm debian date -s 2016-01-01
date: cannot set date: Operation not permitted
$ docker run --rm --cap-add sys_time debian date -s 2016-01-01
date: cannot set date: Operation not permitted
$ docker run --rm --security-opt seccomp:unconfined debian date -s 2016-01-01
date: cannot set date: Operation not permitted
$ docker run --rm --cap-add sys_time --security-opt seccomp:unconfined debian date -s 2016-01-01
Fri Jan  1 00:00:00 UTC 2016
```

[23] https://docs.docker.com/engine/security/seccomp/

[24] harmful means everything that deals with administration (eg: set time) or debugging (eg: ptrace)

[25] https://github.com/moby/moby/blob/master/profiles/seccomp/default.json

User namespaces (rootless mode)

not enabled by default

- UIDs/GIDs inside the containers mapped to another range outside the container
- useful for:
  - let an untrusted user use a docker engine
  - preventing fs-based attacks *(eg: root user inside the container creates a setuid executable in an external volume)*
  - isolating docker users from each other *(one docker daemon for each user, with uids remapped to different ranges)*
- limits
  - global config only (daemon scope)
  - coarse mapping only (hardcoded range: 0..65535)

⚠ Beware of bugs ⚠

Even with all security features enabled, you may still be vulnerable because:

- the kernel is exposed
- the docker internals are quite complex

CVE-2019-5736

runc through 1.0-rc6, as used in Docker before 18.09.2 and other products, allows attackers to overwrite the host runc binary (and consequently obtain host root access)

CVE-2018-15664

In Docker through 18.06.1-ce-rc2, the API endpoints behind the 'docker cp' command are vulnerable to a symlink-exchange attack with Directory Traversal, giving attackers arbitrary read-write access to the host filesystem with root privileges

→ **always keep your docker daemon up-to-date**

Run the docker engine inside a VM

Hypervisors have a smaller attack surface and are much more mature that containers. **Use a VM if you need good isolation!**

- either manually-administrated VMs
- or transparently-launched VMs
  - on a per-engine basis (docker daemon inside a VM)
    docker machine: `https://docs.docker.com/machine/overview/`
  - on a per-container basis (each container in a separate VM)
    kata containers: `https://katacontainers.io/`
    runv: `https://github.com/hyperhq/runv`
    gvisor: `https://github.com/google/gvisor`

Container/Container isolation

- by default all containers can connect to any other container (located in the same bridge)
  - run the daemon with `--icc=false`
    - all communications filtered by default
    - whitelist-based access with `--link`
      (only `EXPOSE`*d ports will be whitelisted)*
  - attach containers to different networks
- by default RAW sockets are enabled (allows ARP spoofing)[26]
  → use `docker run --cap-drop=NET_RAW`

[26] `http://lwn.net/Articles/689453`

Other security considerations

- images are immutable
  → need a process to apply automatic security upgrades, e.g:
  - apply upgrades & commit a new image
  - regenerate the image from the Dockerfile

- docker engine control == root on the host machine
  - give access to the docker socket only to trusted users

- avoid `docker run --privileged` (gives full root access)

- beware of symlinks in external volumes
  eg. ctr1 binds `/data`, ctr2 binds `/data/subdir`, if both are malicious and cooperate, ctr1 replaces `/data/subdir`with a symlink to `/`, then on restart ctr2 has access to the whole host filesystem

  → avoid binding subdirectories, prefer using named volumes

Part 7.
Docker Ecosystem

- Standards
- Docker compose
- Kubernetes

Standards

There are many other container tools for linux, eg: podman, colima, nerdctl, kubernetes (and its declinations like: openshift, rancher, ...)

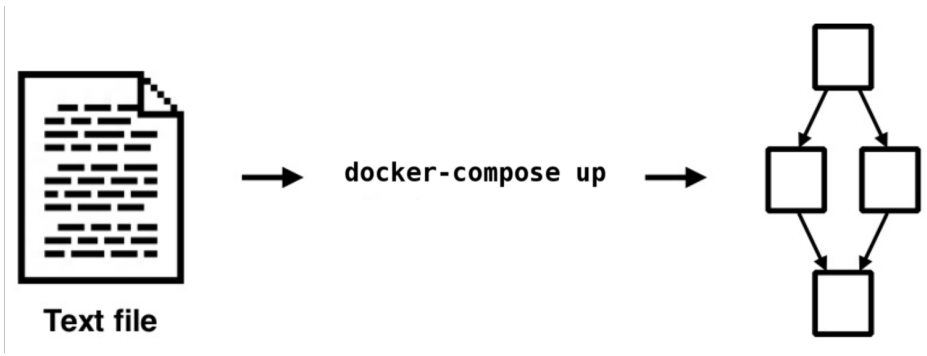
Thankfully they rely on common standards:

- OCI image format and registry API (for image interoperability)
- lower-level tools:
  - buildkit for building the images
  - containerd for managing the containers
  - runc tool for starting the container
- plugin APIs (eg: for implemenent a volume or network provider)
- the compose specification and the Kubernetes API for configuring the containers

Docker Compose

https://docs.docker.com/compose/

A tool and a specification for deploying a collection of containers



group.yml

```
name: counter

containers:
  web:
    build: .
    command: python app.py
    ports:
      - "5000:5000"
    volumes:
      - .:/code
    links:
      - redis
  redis:
    image: redis:latest
```

Kubernetes (k8s)

- project started in 2014 by a group of google developers
- inspired from Google's internal orchestration framework
- now hosted by the Cloud Native Computing Foundation (CNCF) and the de facto standard for deploying containers in a cluster
- declarative API
- large scale, very sophisticated, very versatile, very resilient, very extensible